W8 COS30045

Lecture

Agenda

- 8.1 Accessibility

Class

Agenda

- Demonstration 3
- Team formation
- D3 Accessibility

W8.1 Accessibility

Lecture

Agenda

- WCAG
- Accessible visualisations

References

Dufour & Meeks (2024) Ch 10

Firth (2024) Practical Web Accessibility: A comprehensive guide to digital inclusion Ch 1, 2 and 3

Disabilities that can impact web use

Auditory/Hearing

Cognitive

Physical/Motor

Visual

- Colour vision deficiency, low vision, blindness

Disabilities legislation

- Legal responsibility to make website accessible to people with disability (especially government services)
- Accessible websites can help everyone!

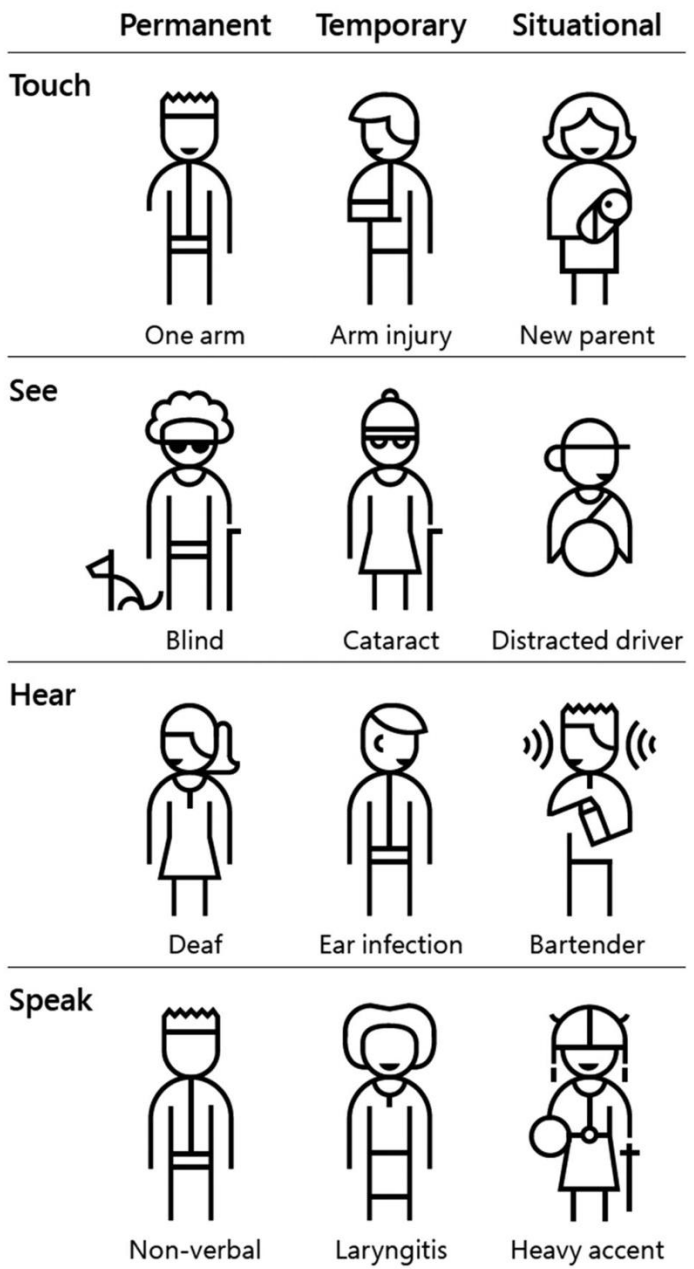


Figure 1-1 A diagram created by Microsoft to showcase how access needs can be permanent, temporary, or situational. They are categorized into touch, see, hear, and speak. The "touch" example shows the permanent disability as someone with one arm, the temporary disability as someone with an arm injury, and the situational disability as a new parent holding a baby.

Firth (2024)

SWINBURNE UNIVERSITY OF TECHNOLOGY

Accessibility - Web Content Accessibility Guidelines (WCAG)

Conformance levels

- **Level A**: Must have. Includes the most basic web accessibility features
  - Alternatives for non-text content, Keyboard accessibility, Seizure prevention
- **Level AA**: Should have. Deals with the biggest and most common barriers for disabled users. It includes all Level A and Level AA criteria.
  - Contrast, Resizable text, navigation, headings and labels
- **Level AAA**: Good to have. The highest and most complex level of web accessibility. It includes all Level A, AA, and AAA criteria
  - Sign language, enhanced contrast, reading level, accessible authentication

SWINBURNE UNIVERSITY OF TECHNOLOGY

Accessibility

Web Content Accessibility Guidelines (WCAG)

- Perceivable
  - Text alternatives
  - Adaptable content
  - Distinguishable
- Operable
  - Keyboard accessibility
  - Enough time
  - Seizure and physical reactions
  - Navigable
- Understandable
  - Readable
  - Predictable
  - Error preventions
- Robust
  - Compatible
  - Valid Code
  - Accessible APIs

SWINBURNE UNIVERSITY OF TECHNOLOGY

Accessing web content with a visual impairment

Adaptive strategies

- Increasing size of text, zooming, reduce mouse speed, access content with keyboard, video captions

Assistive technologies

- Screen readers
- Screen magnifiers
- Speech recognition (voice assistants)
- Switch control (https://support.google.com/accessibility/android/answer/6122836?hl=en)



Challenges making visualisations accessible

Fully appreciating a visualisation requires vision

For people with visual disability:

- Not required to give identical experience
- Give equivalent experience
  - Access to main conclusions of visualisation
  - Access to underlying data

SWINBURNE UNIVERSITY OF TECHNOLOGY

CHARTABILITY

Chartability Critical Heuristics

Perceivable

User must be able to easily identify content using their senses: sight, sound, and touch.

- Low contrast
- Content is only visual
- Small text size
- Visual presents seizure risk



https://chartability.fizz.studio/

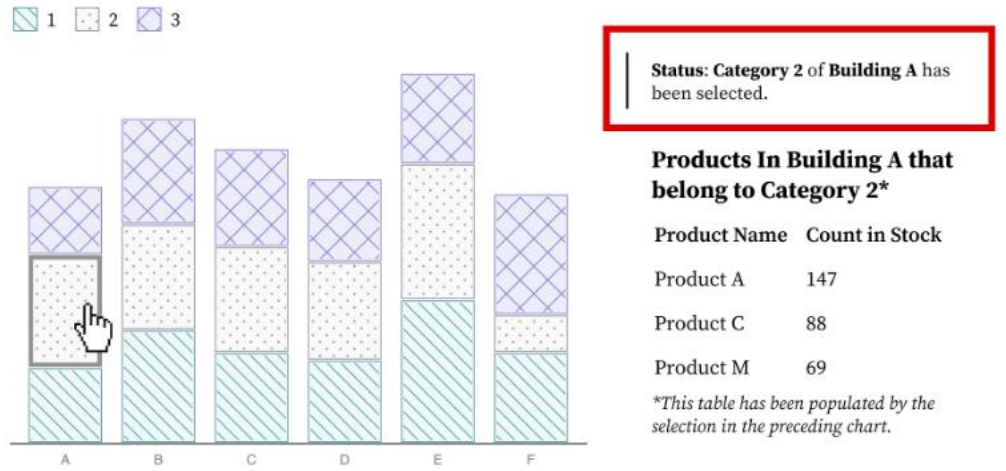
SWINBURNE UNIVERSITY OF TECHNOLOGY

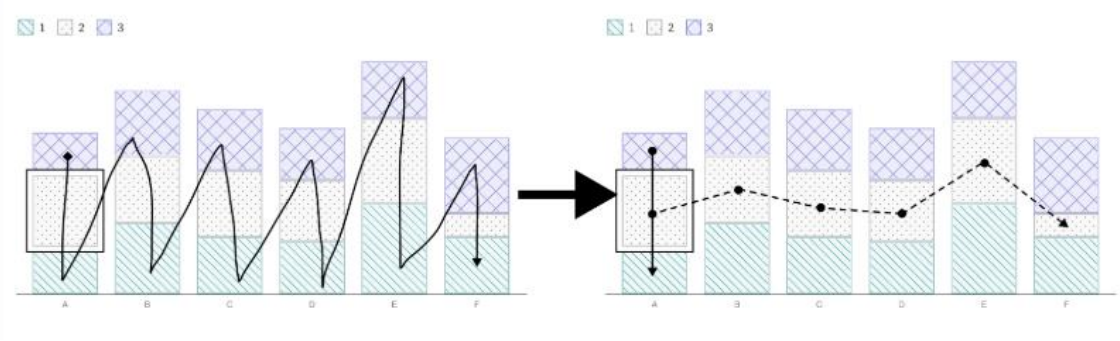
Chartability Critical Heuristics

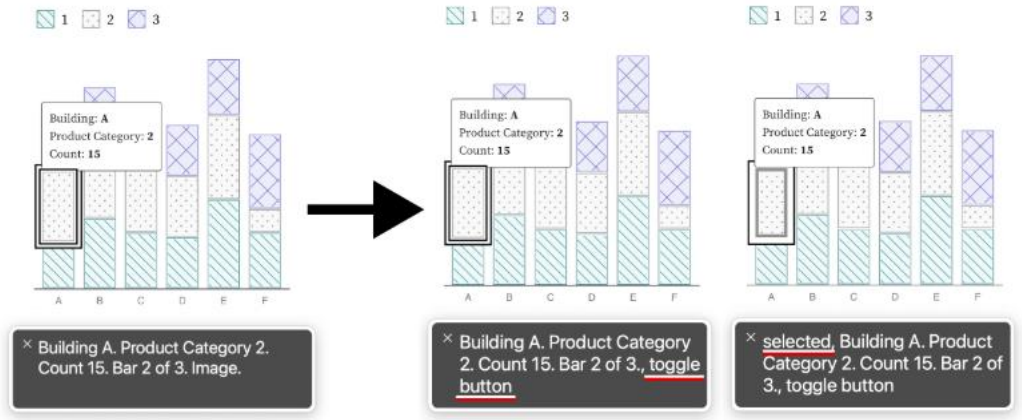
Operable

All controls must be error-tolerant, discoverable, and multi-modal (not just mouse operable, but using keyboard, etc).

- Interaction modality has only one input type
- No interaction cues or instructions
- Controls override Assistive Technology controls







https://chartability.fizz.studio/

CHARTABILITY

Chartability Critical Heuristics

Understandable

Any information or data are presented without ambiguity, with clarity, and in a way that minimizes cognitive load.

- No explanation for purpose or how to read
- No title, summary, or caption
- Reading level inappropriate

https://chartability.fizz.studio/

SWINBURNE UNIVERSITY OF TECHNOLOGY

CHARTABILITY

Chartability Critical Heuristics

Compromising

(Understandable, yet Robust): Information flows must provide transparency, tolerance, and consideration for different ways that users with assistive technologies and disabilities will prefer to consume different information.

- No table



Month	Registered Clients
Jan	10
Feb	15
Mar	12
Apr	16
May	18
Jun	23
Jul	34
Aug	51
Sept	86
Oct	143
Nov	228
Dec	386

https://chartability.fizz.studio/

SWINBURNE UNIVERSITY OF TECHNOLOGY

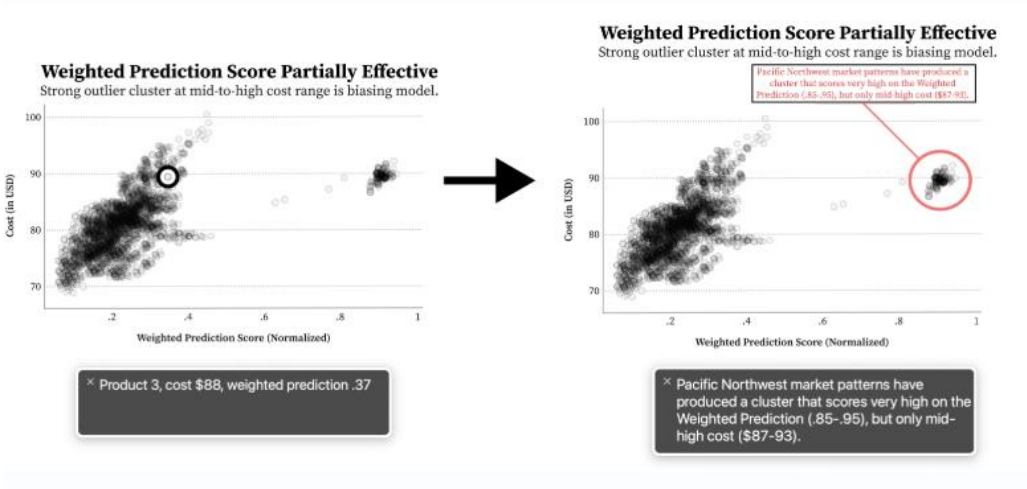
CHARTABILITY

Chartability Critical Heuristics

Assistive

(Understandable and Perceivable but labor-reducing): Interface must be intelligent and multi-sensory in a way that reduces the cognitive and functional labor required for use.

- Data density is inappropriate
- Navigation and interaction is tedious



Flexible

(Perceivable and Operable, yet Robust): Design must respect user settings from user agents (browsers, operating systems, applications) and provide presentation and operation control.

- User style change not respected

https://chartability.fizz.studio/

SWINBURNE UNIVERSITY OF TECHNOLOGY

Accessibility Checklist

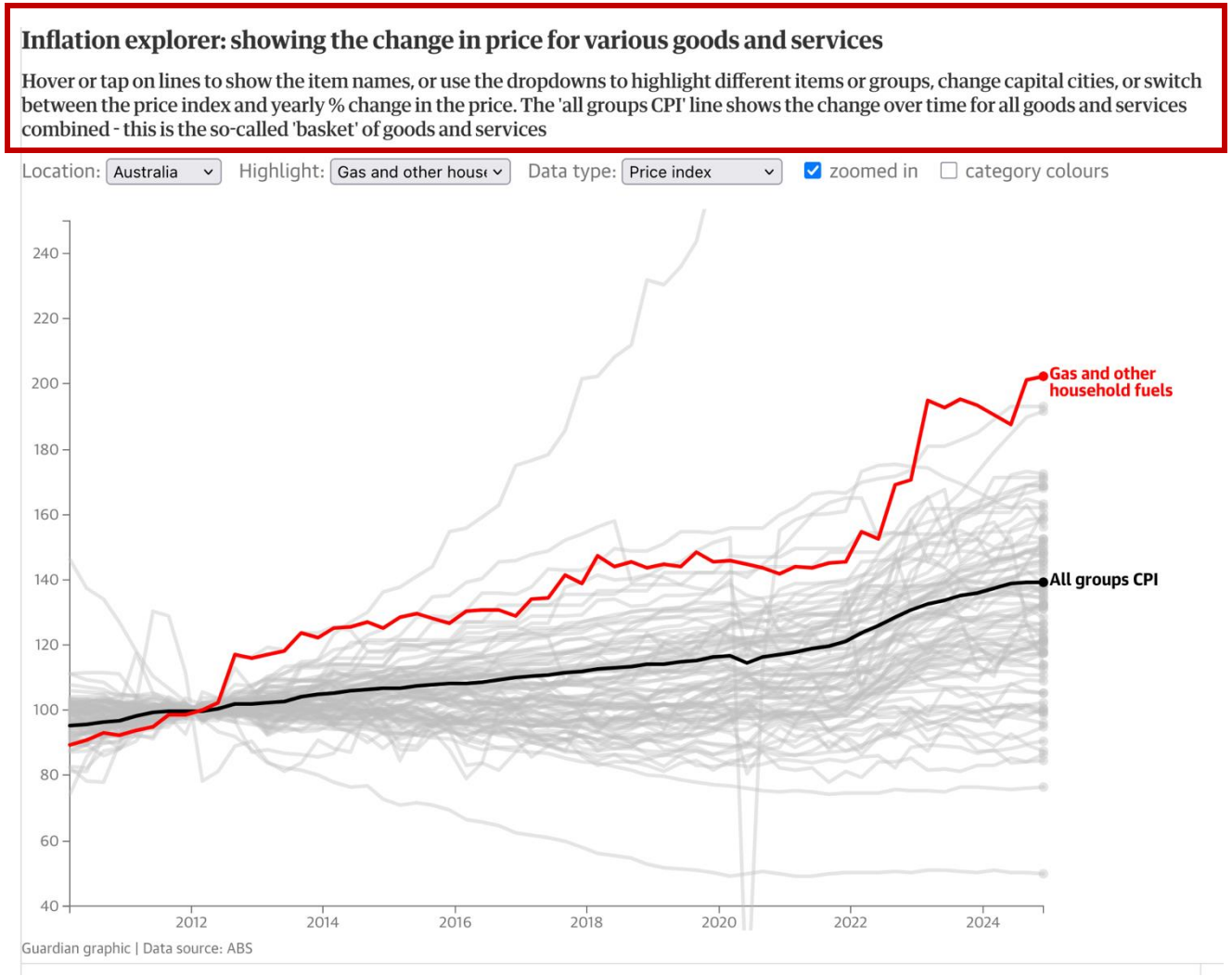
- Textual information
- Visual information
- Screen readers access
- Interactions
- Other considerations

Dufour & Meeks (2024)

Textual

Purpose and how to read chart explained

- Introduction section
- Legend



https://www.theguardian.com/news/datablog/ng-interactive/2023/apr/26/cost-of-living-2023-see-how-inflation-has-changed-prices-in-australia-in-the-march-quarter-interactive

SWINBURNE UNIVERSITY OF TECHNOLOGY

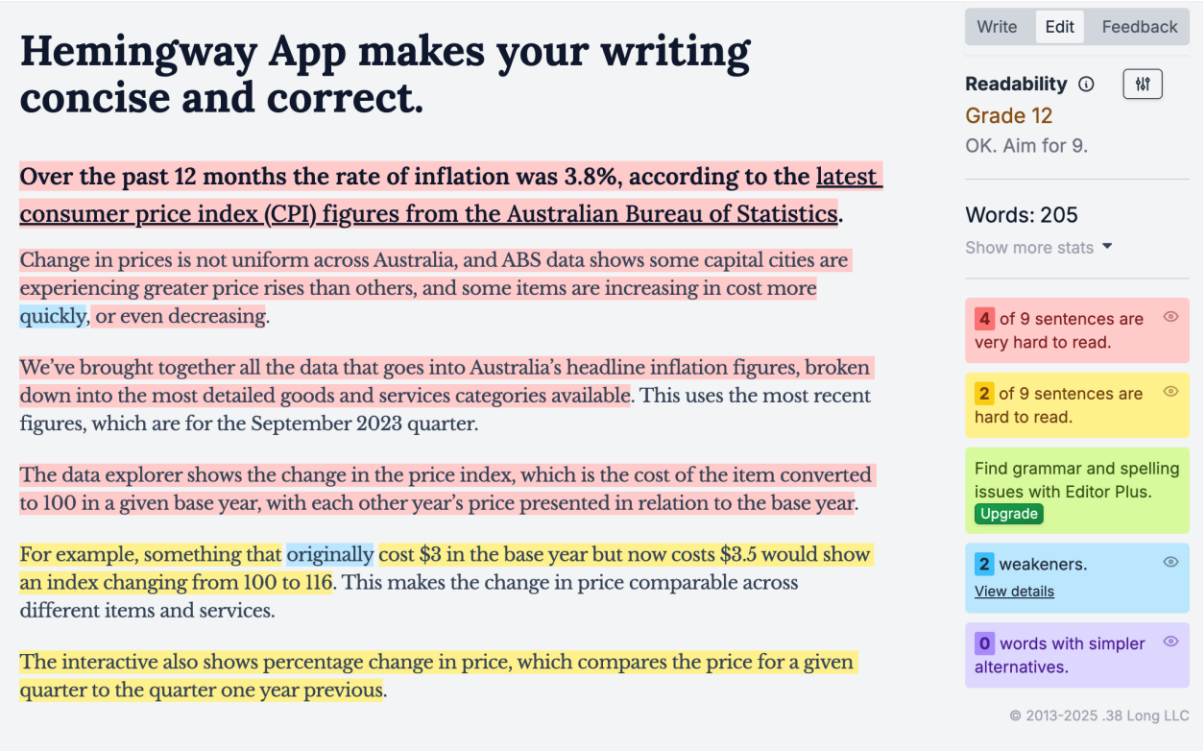
Textual

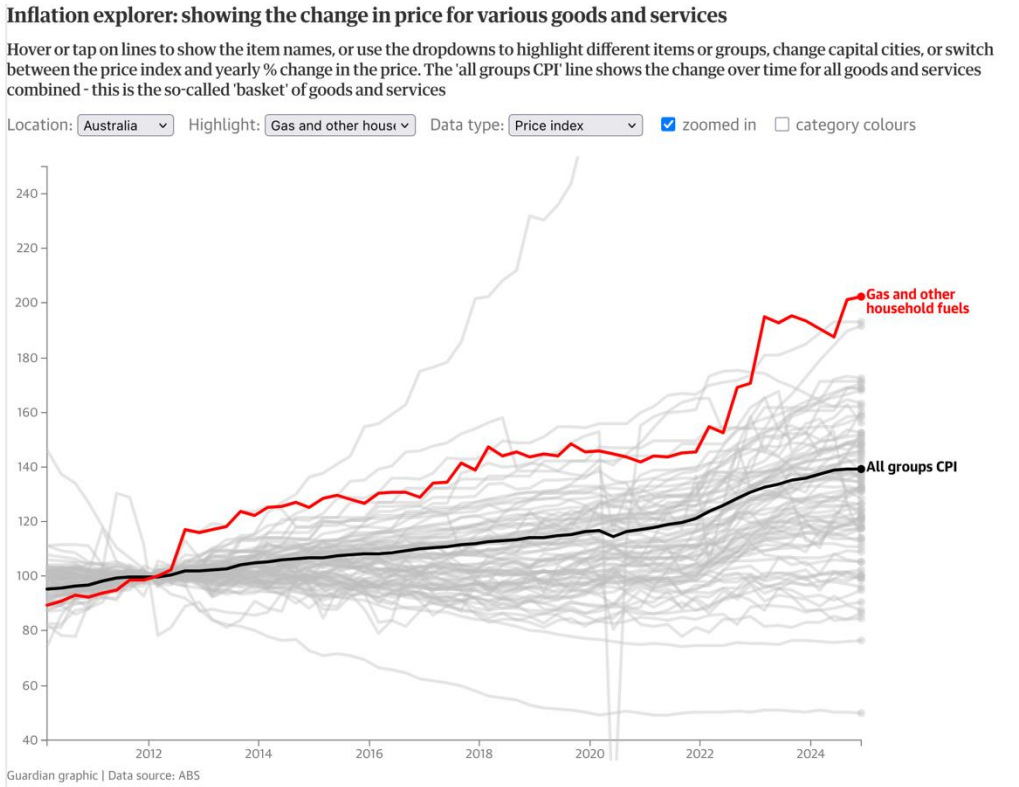
Information reading grade level

For general public users:

- Check that reading level is at level 9 or lower

  https://hemingwayapp.com/





https://www.theguardian.com/news/datablog/ng-interactive/2023/apr/26/cost-of-living-2023-see-how-inflation-has-changed-prices-in-australia-in-the-march-quarter-interactive

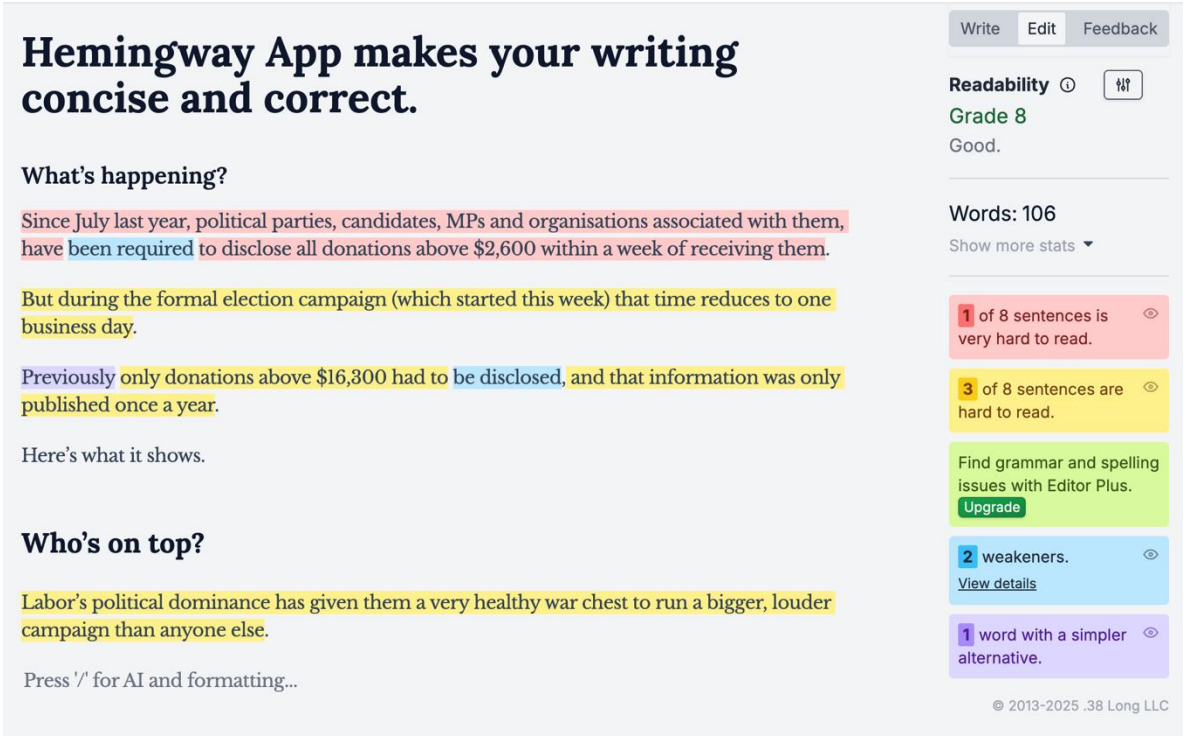
Textual

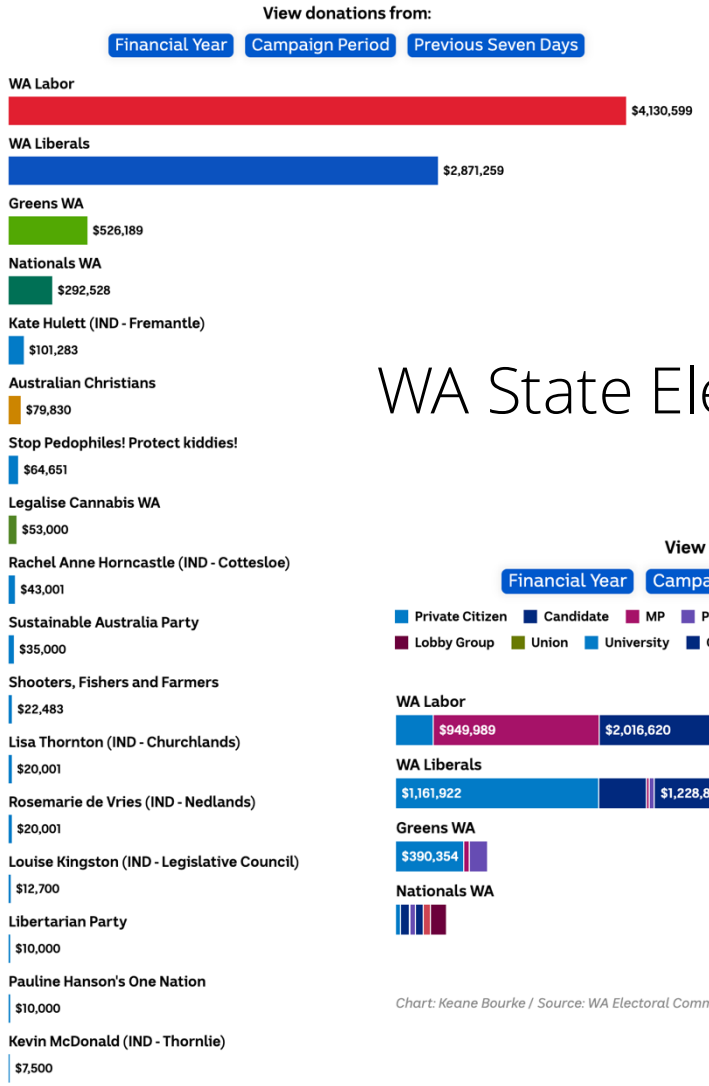
Information reading grade level

For general public users:

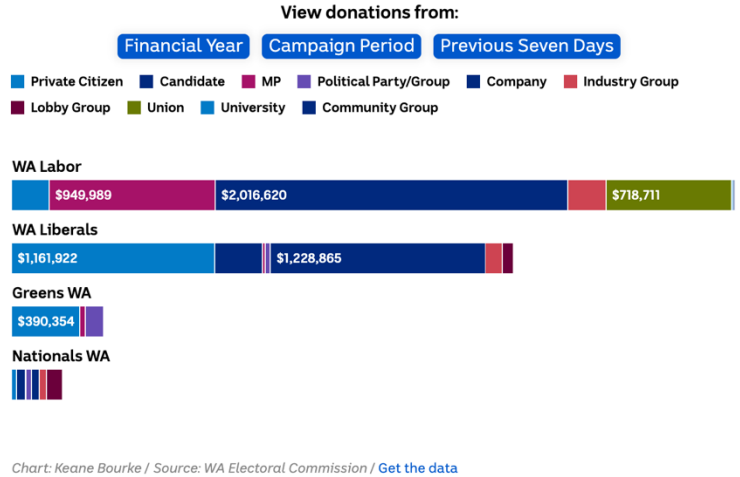
- Check that reading level is at level 9 or lower

  https://hemingwayapp.com/





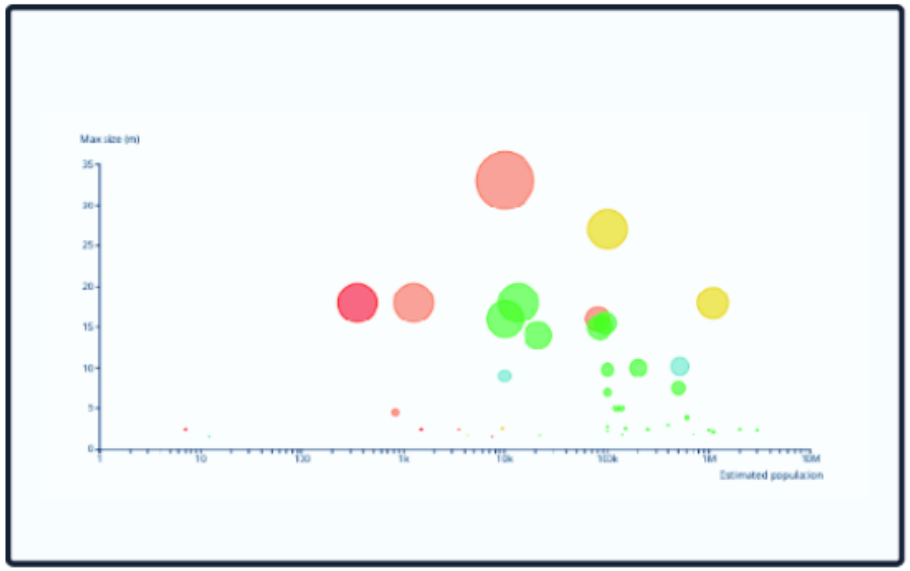
WA State Elections(2025)



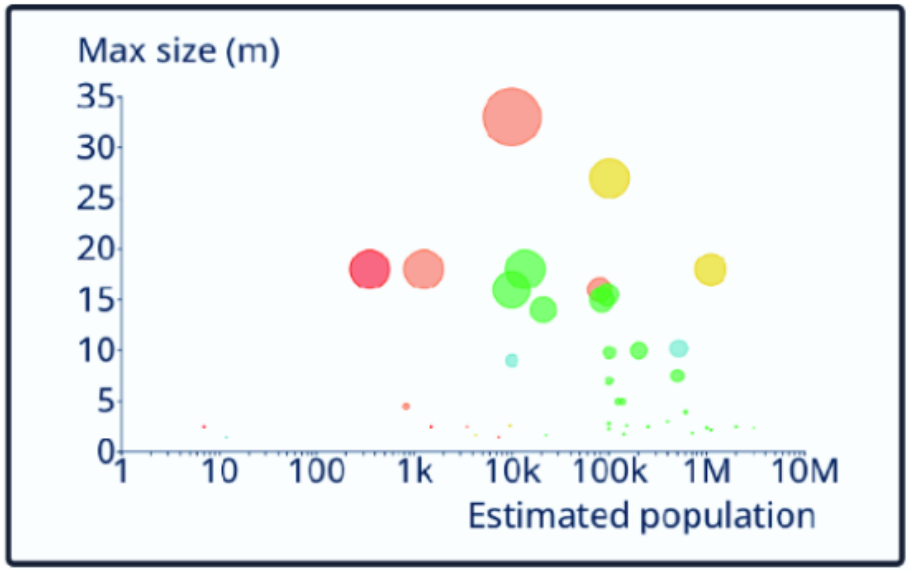
https://www.abc.net.au/news/2025-02-06/wa-election-live-political-donations-tracker-2025-/104573748?utm_source=abc_news_app&utm_medium=content_shared&utm_campaign=abc_news_app&utm_content=mail

SWINBURNE UNIVERSITY OF TECHNOLOGY

Responsive text size



Before



After

Figure 10.6 Because the text acts like any other graphical element, the axis labels looked tiny on small screens. With the help of a linear scale, the text is dynamically resized based on the screen's width.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Text size

Chart text size should be at least 12px

(other text should be at least 16px)

Reducing screen size will reduce text size

Rule of three:

A ~ B (max svg width (1200) - small svg width (315))

C ~ X (svg font – small svg font)

X = B × C ÷ A

How big will our small svg container font be?

315 × 15 ÷ 1,200 ≈ 4

How big will our max svg font need to be so scales to 15 px on small screen?

15 × 1,200 ÷ 315 ≈ 57

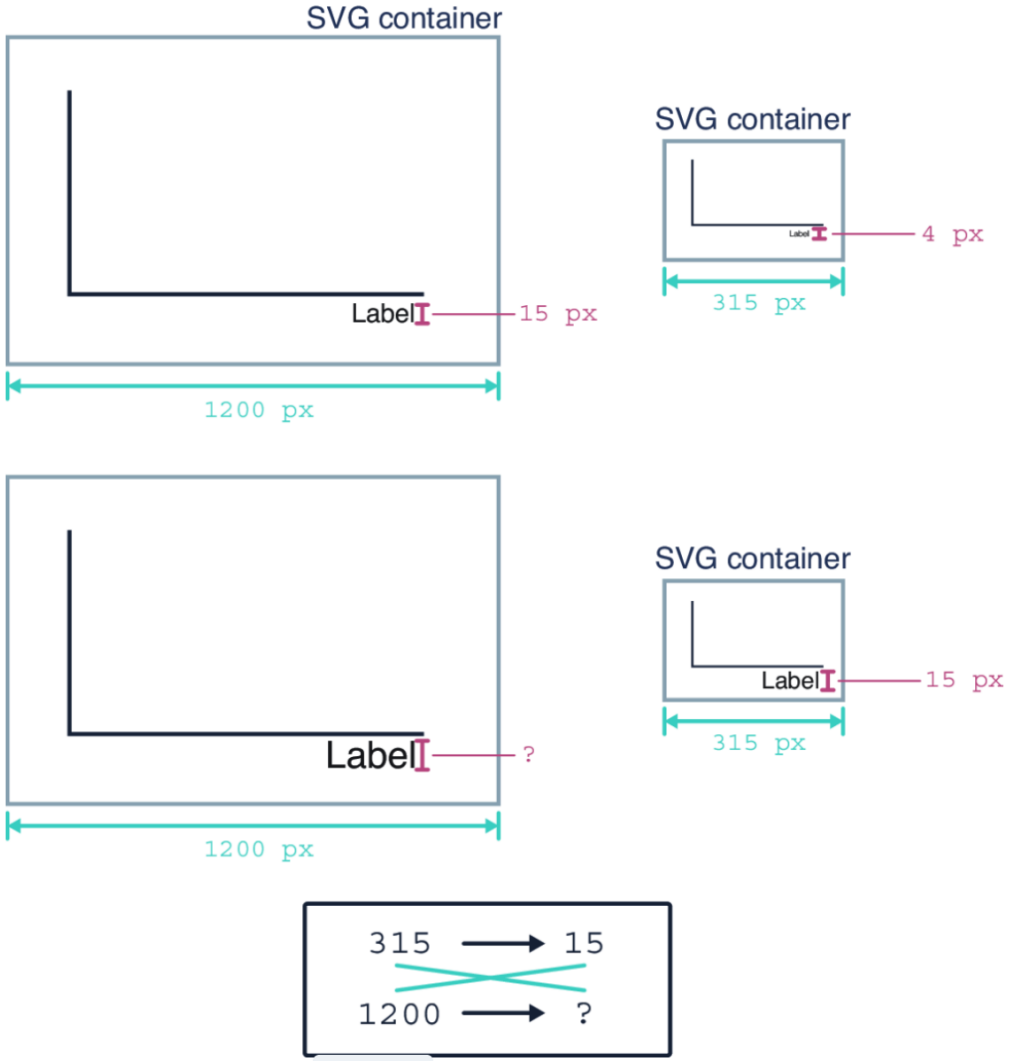


Figure 10.5 Because the `viewBox` attribute of the SVG container is set and its `width` and `height` attributes are not, all the graphical elements resize with the screen. If the font size of the axis label is set to 15 px, they will look much smaller on phone screens. We can use a rule of three to find the actual font size that we need to apply on phone screens.

shared-constants.js

Add a function to scale font size relative to min and max screen widths

```
/*******************************/
/*    Declare the constants    */
/*******************************/
const margin = {top: 40, right: 30, bottom: 50, left: 40};
const width = 1000;
const height = 500;
const innerWidth = width - margin.left - margin.right;
const innerHeight = height - margin.top - margin.bottom;
```

May need to adjust margins to fit axis labels nicely

```
const fontSizeScale = d3.scaleLinear()
  .domain([315, 1000])
  .range([47, 15])
  .clamp(true);
```

15 × 1,000 ÷ 315 ≈ 47

30 seemed to work best… on my system

SWINBURNE UNIVERSITY OF TECHNOLOGY

resize.js

Don't forget to add resize.js to index.html

```
const resizeChart = () => {

    windowWidth = window.innerWidth;

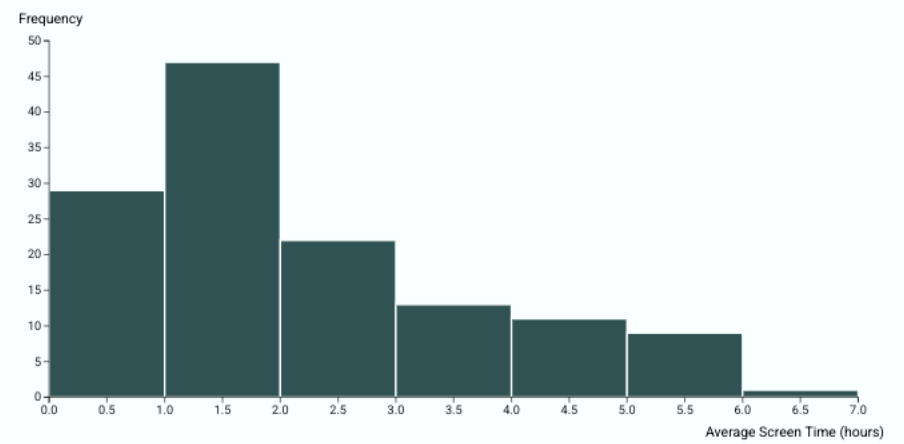
    // Resize axis labels
    d3.selectAll(".tick text, .axis-label")
        .style("font-size", `${fontSizeScale(windowWidth)}px`);
    d3.select(".axis-label-left")
        .attr("y", margin.top - fontSizeScale(windowWidth));
    d3.select(".axis-label-bottom")
        .attr("y", height - margin.bottom + 2.2*fontSizeScale(windowWidth));
};

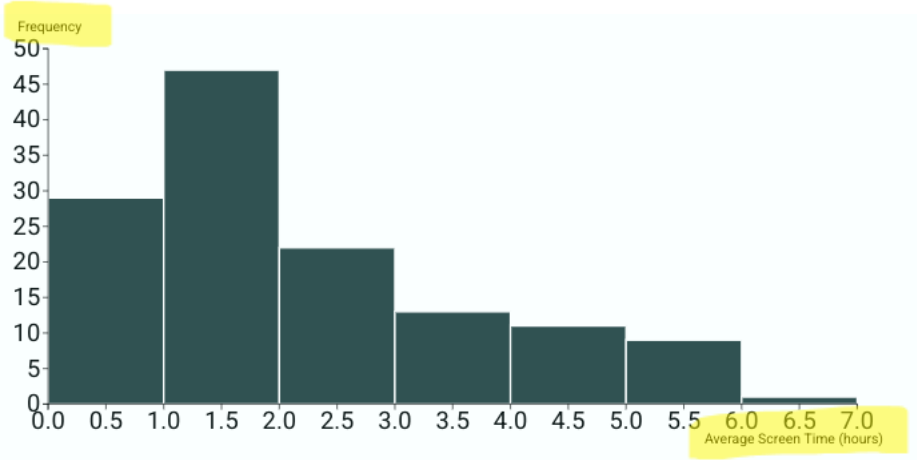
window.addEventListener("resize", () => {
    resizeChart();
});
```

Call **resizeChart()** in chart function so it will automatically resize, otherwise it will only update when triggered (i.e., when user changes window size)

SWINBURNE UNIVERSITY OF TECHNOLOGY

histogram.js





Add class to axis labels (**histogram.js**)

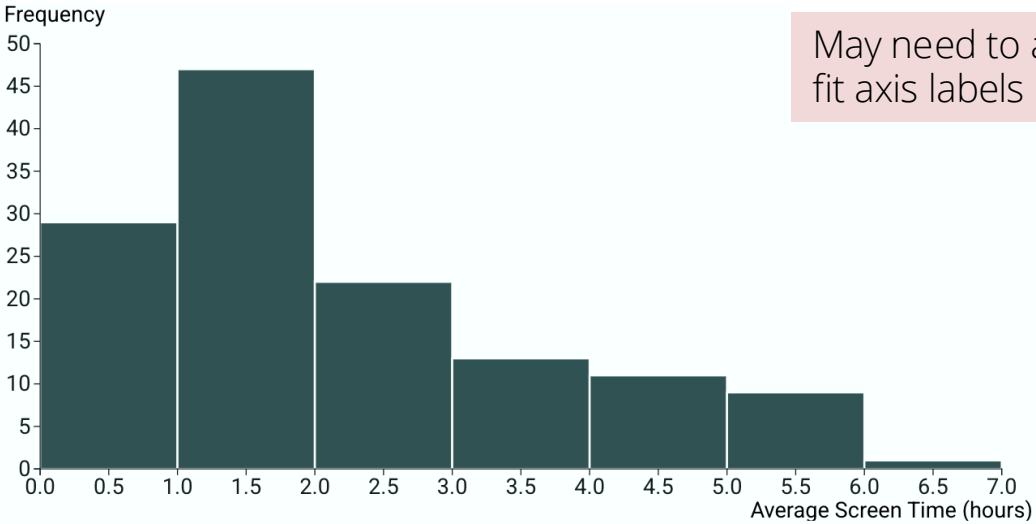
```
// Add the x-axis label
  svg
    .append("text")
      .text("Average Screen Time (hours)")
      .attr("class", "axis-label axis-label-bottom")
      .attr("text-anchor", "end")
      .attr("x", width)
      .attr("y", height - 5);

// construct the x-axis
  const leftAxis = d3.axisLeft(yScale);

  // Add the y-axis to the left of the chart relative to the inner chart
  innerChart
    .append("g")
      .call(leftAxis);

// Add the y-axis label
  svg
    .append("text")
      .text("Frequency")
      .attr("class", "axis-label axis-label-left")
      .attr("x", 5)
      .attr("y", 20);
```

```
**********/
eclare the constants    */
**********/
argin = {top: 50, right: 30, bottom: 60, left: 40};
idth = 1000;
eight = 500;
nnerWidth = width - margin.left - margin.right;
nnerHeight = height - margin.top - margin.bottom;
```



May need to adjust margins to fit axis labels nicely

SWINBURNE UNIVERSITY OF TECHNOLOGY

Contrast

Contrast ratio

- Geometric shapes – 2 to 1
- Regular text – 4.5 to 1

Check with accessibility checkers

Background \ Text	#FFFFFF	#F2F2F2	#DDDDDD	#CCCCCC	#888888	#404040	#000000	#2F78C5	#0F60B6	#398EEA
White #FFFFFF		DNP 1.1	DNP 1.3	DNP 1.6	AA18 3.5	AAA 10	AAA 21	AA 4.5	AA 6.2	AA18 3.3
#F2F2F2	DNP 1.1		DNP 1.2	DNP 1.4	AA18 3.1	AAA 9.2	AAA 18.7	AA18 4	AA 5.5	AA18 3
#DDDDDD	DNP 1.3	DNP 1.2		DNP 1.1	DNP 2.6	AAA 7.6	AAA 15.4	AA18 3.3	AA 4.5	DNP 2.4
#CCCCCC	DNP 1.6	DNP 1.4	DNP 1.1		DNP 2.2	AA 6.4	AAA 13	DNP 2.8	AA18 3.8	DNP 2.1
#888888	AA18 3.5	AA18 3.1	DNP 2.6	DNP 2.2		DNP 2.9	AA 5.9	DNP 1.2	DNP 1.7	DNP 1
Charcoal #404040	AAA 10	AAA 9.2	AAA 7.6	AA 6.4	DNP 2.9		DNP 2	DNP 2.2	DNP 1.6	AA18 3
Black #000000	AAA 21	AAA 18.7	AAA 15.4	AAA 13	AA 5.9	DNP 2		AA 4.6	AA18 3.3	AA 6.2
Effective on Extremes #2F78C5	AA 4.5	AA18 4	AA18 3.3	DNP 2.8	DNP 1.2	DNP 2.2	AA 4.6		DNP 1.3	DNP 1.3
Effective on Lights #0F60B6	AA 6.2	AA 5.5	AA 4.5	AA18 3.8	DNP 1.7	DNP 1.6	AA18 3.3	DNP 1.3		DNP 1.8
Ineffective #398EEA	AA18 3.3	AA18 3	DNP 2.4	DNP 2.1	DNP 1	AA18 3	AA 6.2	DNP 1.3	DNP 1.8	

AAA Pass, AAA (7+)
AA Pass, AA (4.5+)
AA18 Pass, Large Text Only (3+)
DNP Does Not Pass

About WCAG 2.0 contrast

[Contrast Grid]

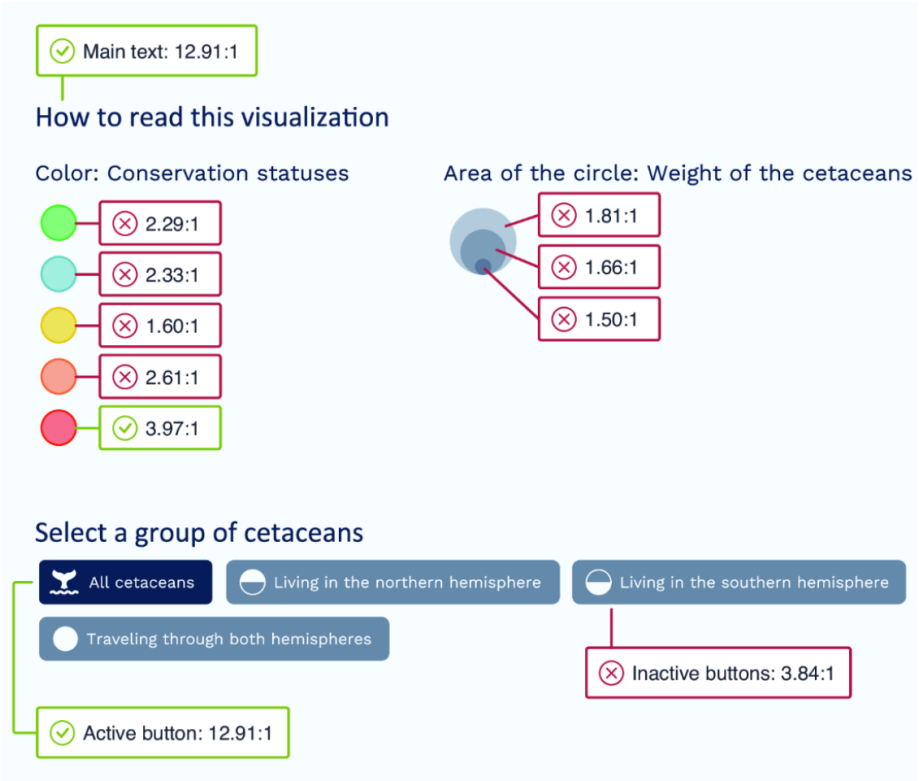


Figure 10.7 According to the WCAG Color contrast checker extension, the text and active button on our chart present sufficient contrast. But the contrast of the text on the inactive buttons isn't high enough to meet the standards. The colors selected to represent the conservation statuses and the legend for the area of the circle all fail to create a sufficient contrast with their background.

contrast.



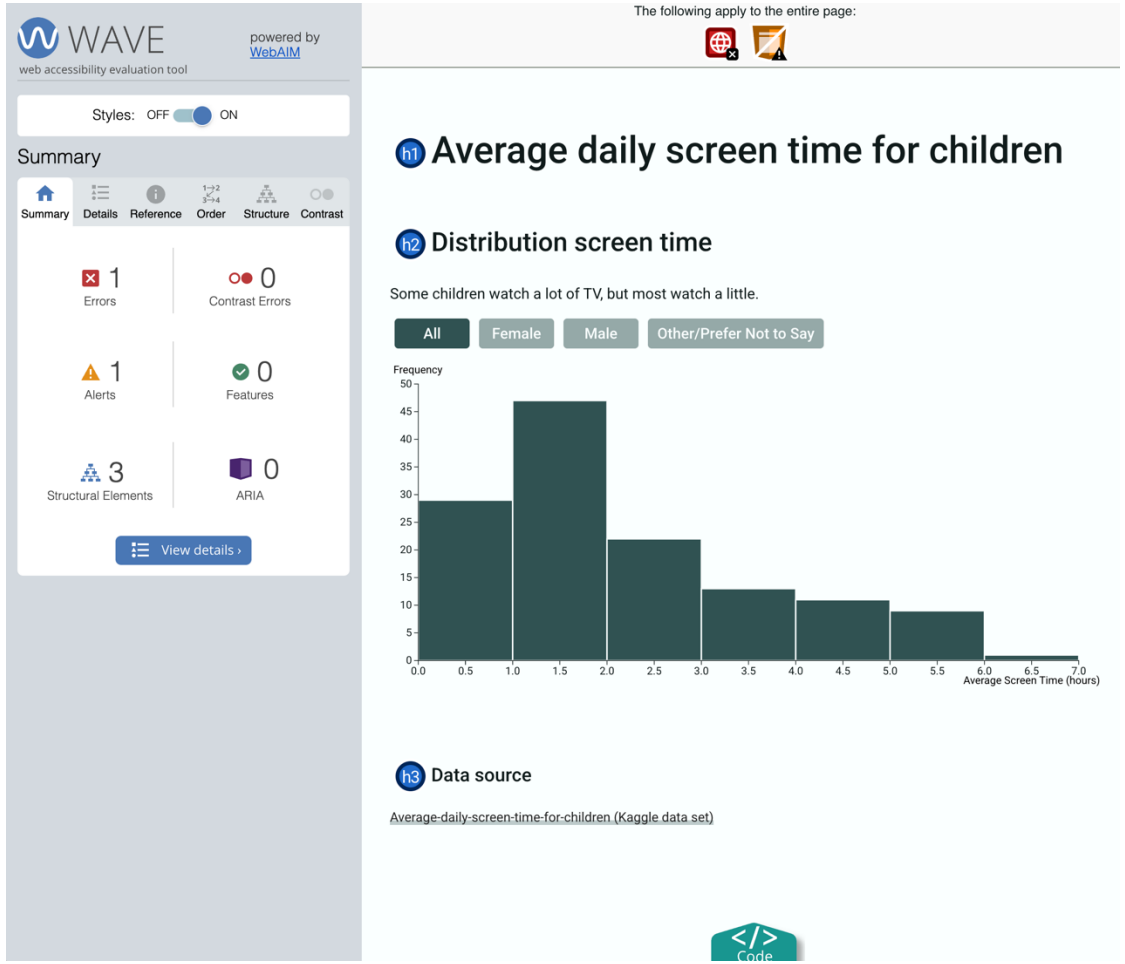
Figure 10.8 The new colors associated with the conservation statuses offer sufficient contrast against a white background, which improves accessibility.

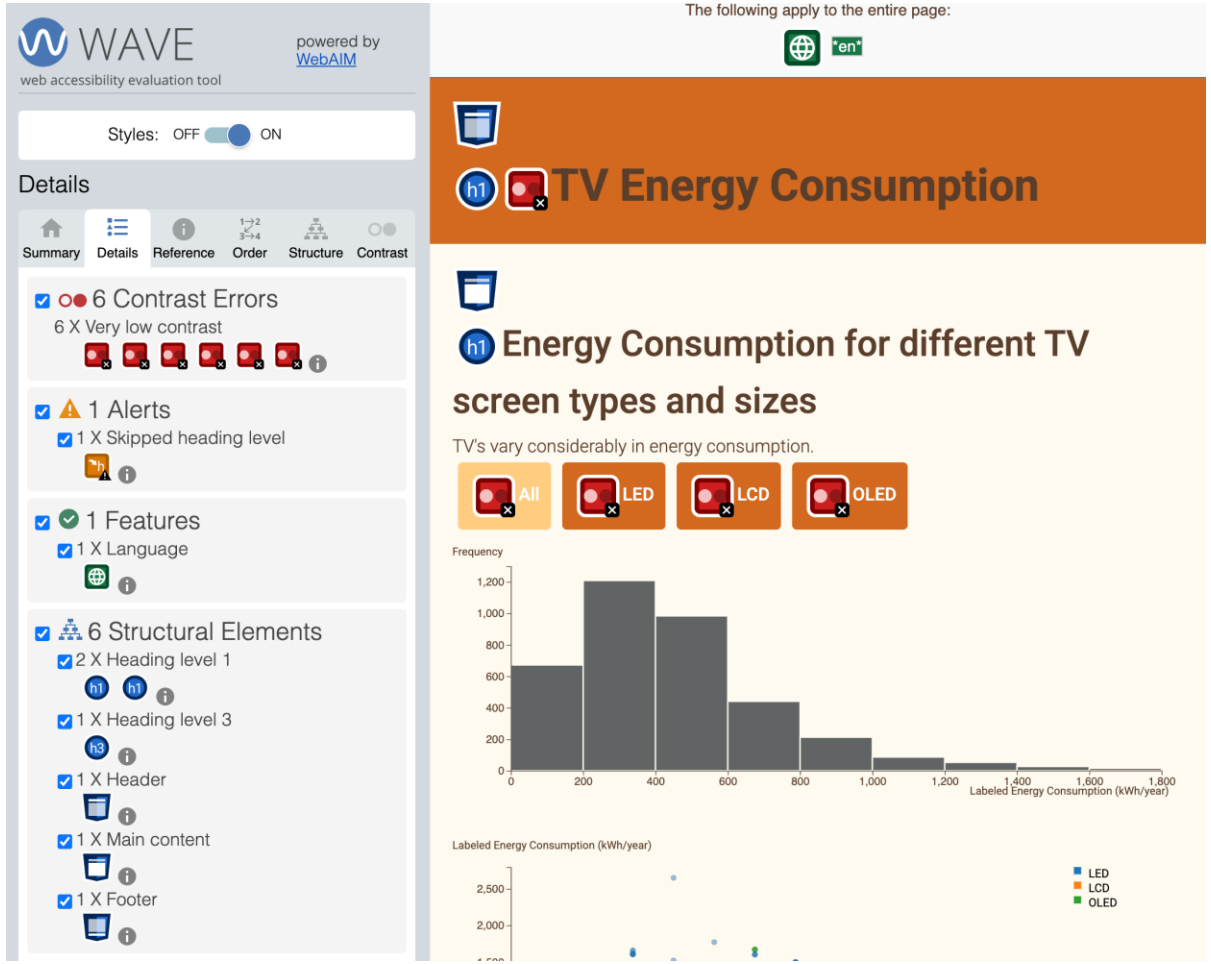
Dufour & Meeks (2024)

[USA gov Data Visualization Standards]

SWINBURNE UNIVERSITY OF TECHNOLOGY

WAVE extension (WebAIM)





SWINBURNE UNIVERSITY OF TECHNOLOGY

Tools for checking contrast

Contrast checking

- Adobe Colour Contrast Checker (https://color.adobe.com/create/color-contrast-analyzer)
- WebAIM (https://webaim.org/resources/contrastchecker/)
- TPGi (app) (https://www.tpgi.com/color-contrast-checker/)
- Contrast Grid
- Accessible color palette builder (https://toolness.github.io/accessible-color-matrix/)

Browser extensions

- WebAIM (https://wave.webaim.org/extension/)

Dev Tools

- Lighthouse (Chrome https://developer.chrome.com/docs/lighthouse/)

SWINBURNE UNIVERSITY OF TECHNOLOGY

Colour blindness

Double Encoding

Colour plus something else

- Different shape
- Different pattern

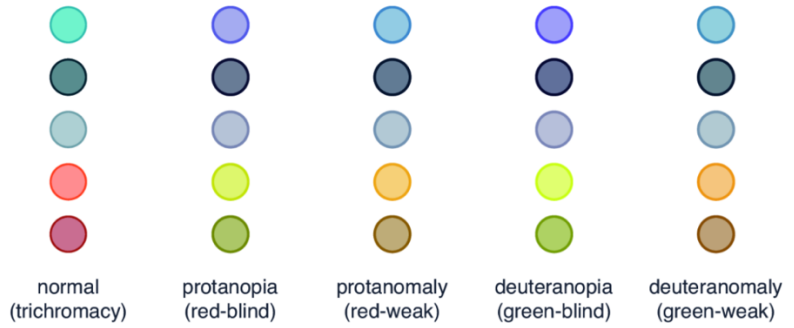
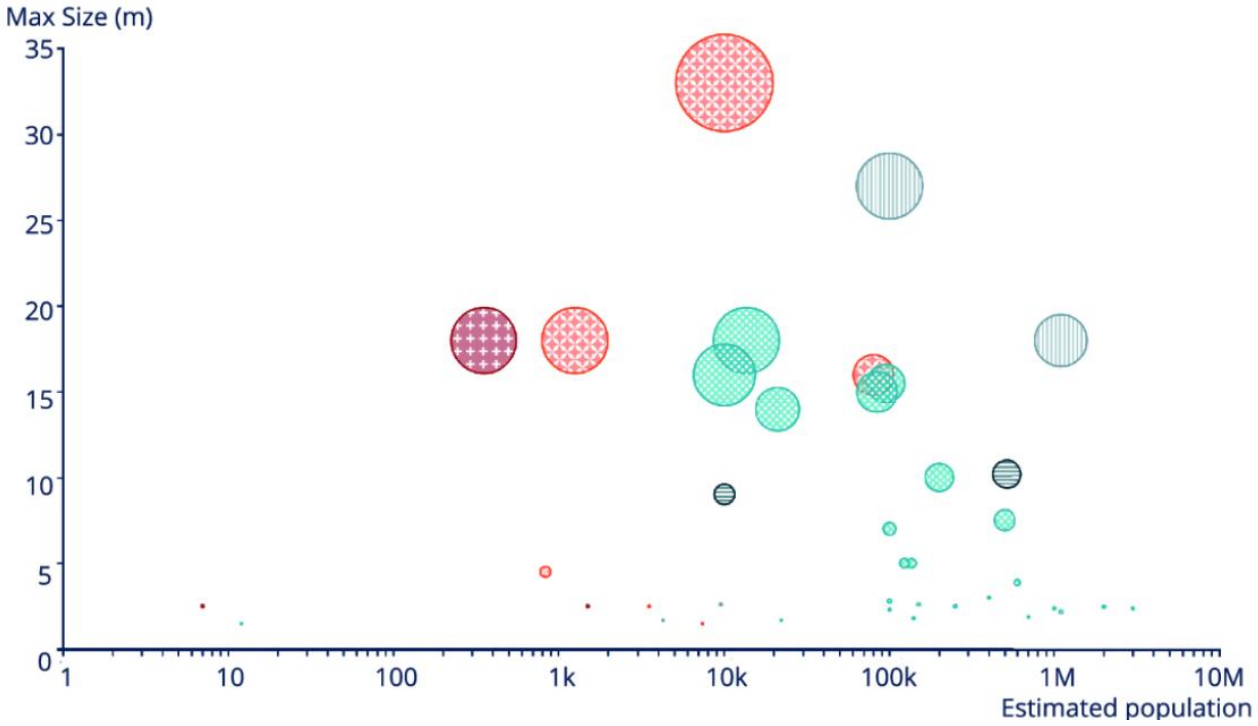




Figure 10.12 We'll apply a different pattern to each conservation status. Combining a color with a pattern is an example of the double encoding strategy.



Durfor &v Meeks (2024)

Data Density

Too much overlapping data

- Explanation of cluster
- Aggregate to make fewer elements

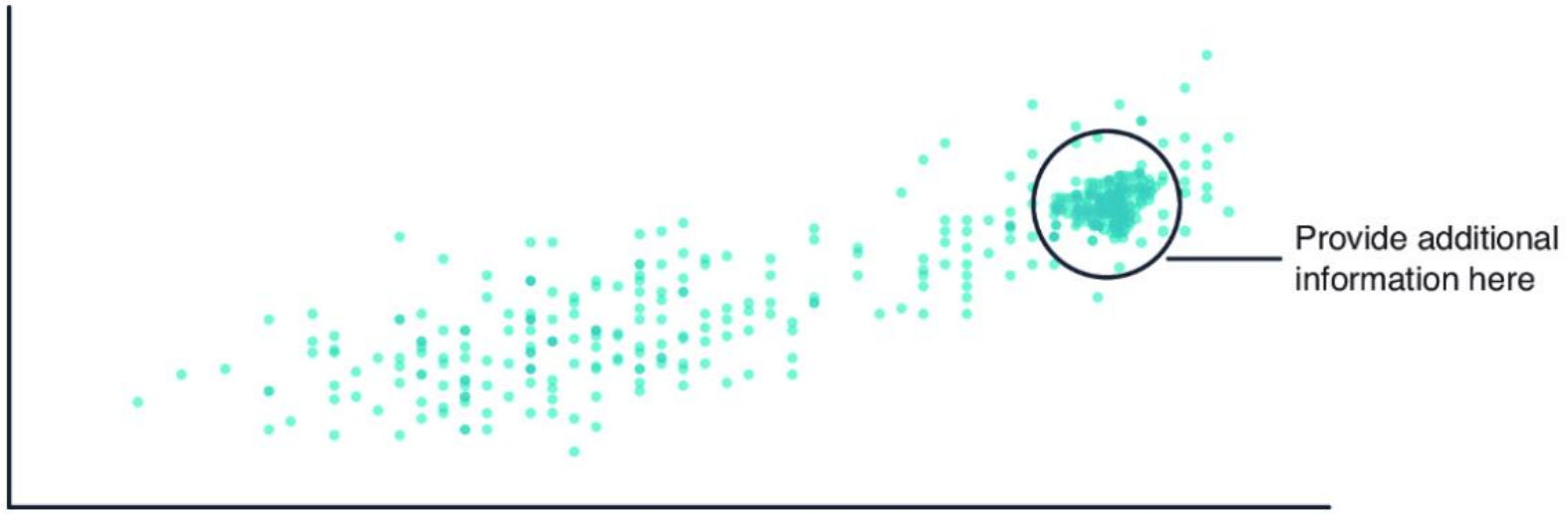


Figure 10.14 When too many elements compete for the same space on a data visualization, it's recommended to provide additional information that explains the pattern or to aggregate the chart to a level with fewer elements.

Durfor &v Meeks (2024)

Screen readers

Users need to access information in visualisation

Does not need to be identical experience

Common screen readers:

- Narrator
- VoiceOver
- ChromeVox
- Jaws….

To experience how it works… Install ChromeVox extension

SWINBURNE UNIVERSITY OF TECHNOLOGY

Screen readers

Semantic tags

HTML tags to guide how screen reader reads content to users. For example,

`<title>`

`<head>`

`<h1>`

Navigation

Via click or keyboard navigation (e.g. tab)

SWINBURNE UNIVERSITY OF TECHNOLOGY

Semantic markup

Heading elements

- E.g., <h1> <h2> <h3>
- Should only have one <h1> element

Logic of page

- <header>, <nav>, <main>, <article>, <section>, and <footer>

Understand navigation

- <buttons>, <a>, <form>

Grouping collection of items

- ,

SWINBURNE UNIVERSITY OF TECHNOLOGY

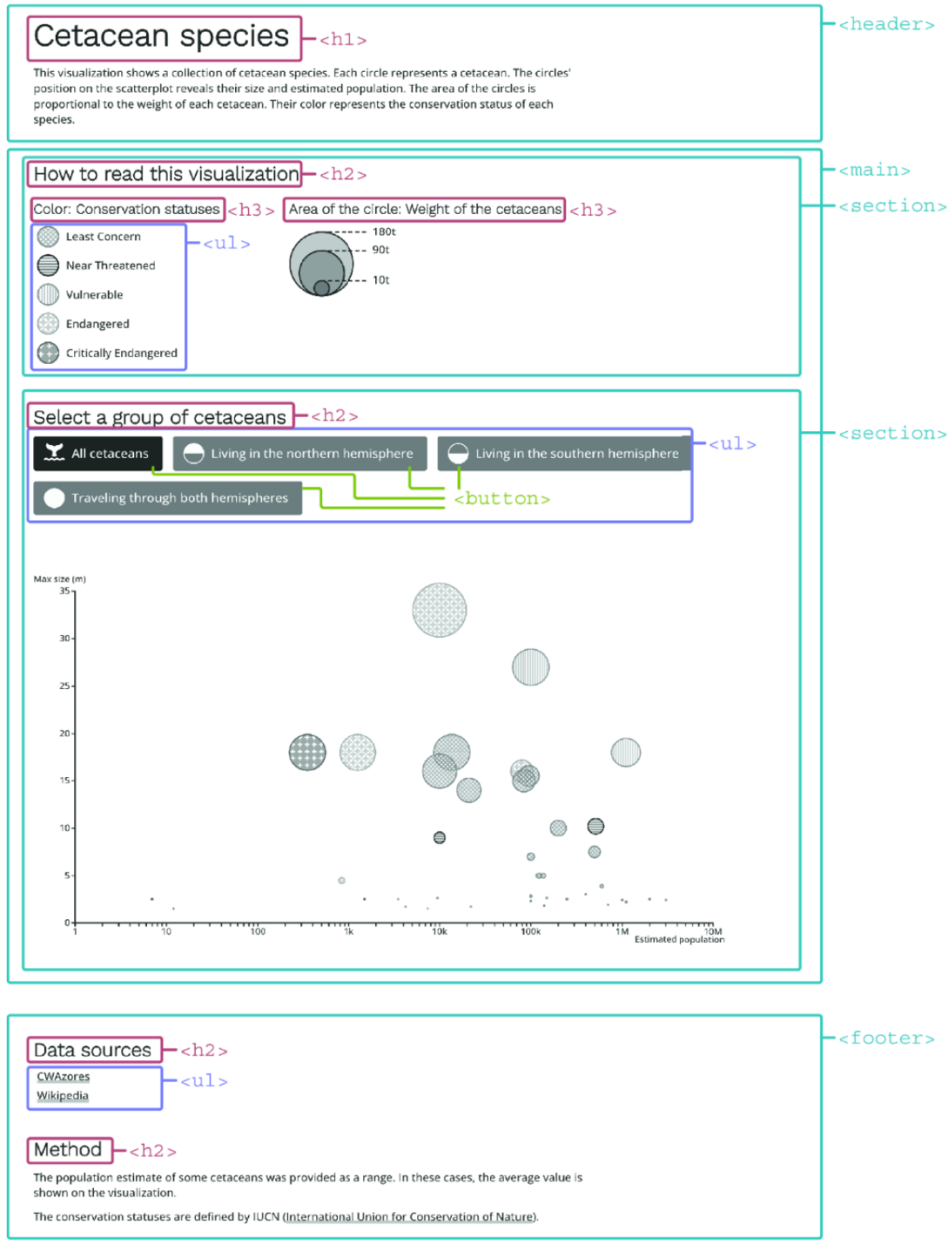


Figure 10.15 Proper semantic HTML elements help screen reader users understand the page's layout and are great for SEO.

Help screen reader interpret set of buttons

Use ARIA (Accessible Rich Internet Applications) to define buttons as a group

Add ARIA label to let screen reader know what they are for

```
d3.select("#filters_gender")
  .attr("role", "group")
  .attr("aria-label", "Filter by gender")
```

role attribute specifically to help assistive technologies. Example:

Button, group, alert, dialog, menu, tooltip, progress bar, img

SWINBURNE UNIVERSITY OF TECHNOLOGY

Help screen reader interpret set of buttons

Add type label to let screen reader know it is a generic button (not a submit or reset button)

Add aria attribute to let screen reader know toggle state (true or false)

```
…
.selectAll(".filter")
.data(filters_gender)
.join("button")
    .attr("type", "button")
    .attr("class", d => `filter ${d.isActive ? "active" : ""}`)
    .attr("aria-pressed", d => d.isActive)
    .text(d => d.label)
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

Help screen reader interpret set of buttons

Update state of button with ARIA

```
.on("click", (e, d) => {

    if (!d.isActive) {
        filters_gender.forEach(filter => {
                filter.isActive = d.id === filter.id;
    });

    d3.selectAll("#filters_gender .filter")
        .classed("active", filter => filter.isActive)
        .attr("aria-pressed", filter => filter.isActive);
```

```
const filters_gender = [
    { id: "all", label: "All", isActive: true },
    { id: "female", label: "Female", isActive: false },
    { id: "male", label: "Male", isActive: false },
    { id: "other_prefer_not_to_say", label: "Other/Prefer Not to Say", isActive: false }
];
```

SWINBURNE UNIVERSITY OF TECHNOLOGY

SVG container

Screen readers can read axis labels

But no other information

Provide meta data for visualization

Add meta data tags to the SVG container:

- <title> and
- <desc> (for description of SVG)

Screen reader reads out to user

SWINBURNE UNIVERSITY OF TECHNOLOGY

Help screen reader provide information about content of SVG

Add role attribute **img** :

- Lets screen reader know that image
- Stop screen reader reading text in SVG inappropriately (i.e., reading out all tick labels)

Add ARIA attribute

```
const svg = d3.select("#histogram")
    .append("svg")
    .attr("viewBox", `0, 0, ${width}, ${height}`)
    .attr("role", "img")
    .attr("aria-labelledby", "histogramTitle histogramDescription");
```

. . . . . .

. . . . . .

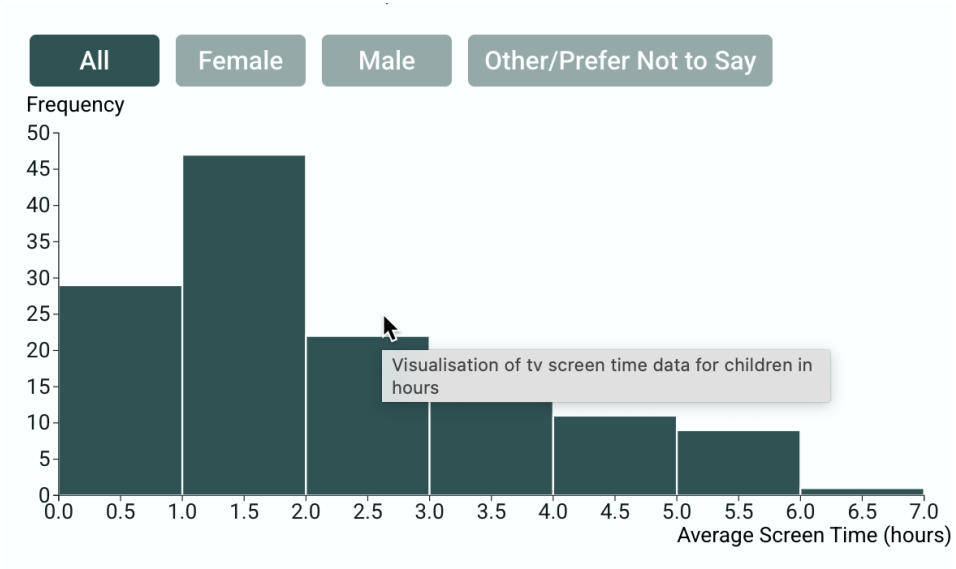
SWINBURNE UNIVERSITY OF TECHNOLOGY

Add a short **title** to SVG

Read by screen reader

Enable user to determine if chart is of interest to them

Title will appear as tool tip




```
svg
    .append("title")
    .attr("id", "histogramTitle")
    .text("Visualisation of tv screen time data for children in hours");
```

Must be added to main SVG container, not inner chart (if using)

SWINBURNE UNIVERSITY OF TECHNOLOGY

Add detailed description to SVG

Description for screen reader

Includes summary of information contained in chart

```
svg
    .append("desc")
    .attr("id", "scatterplotDescription")
    .text(”This is a histogram plotting frequency of hours of
tv watched by children. Most of the children sampled spend
between 1 to 2 hours a day watching TV. A bit over half of
that amount watched between 0 and 1 hour a day. The hours
watched steady decreases with a only a few children
watching 7 hours per day.");
```

. . . . . .

. . . . . .

SWINBURNE UNIVERSITY OF TECHNOLOGY

Add description of data points (to chart file)

```
const generateDescription = (bins) => {

const desc = [];
    // Add introductory text
    desc.push("This histogram visualizes the frequency of hours children spend watching
    TV.");
    // Add details for each bin
    bins.forEach((bin, index) => {
        const rangeStart = bin.x0; // Start of the bin range
        const rangeEnd = bin.x1; // End of the bin range
        const count = bin.length; // Number of children in this bin
        // Generate a description for the current bin
        const binDescription = `${rangeStart} to ${rangeEnd} hours: ${count} children.`;
        desc.push(binDescription);
    });
// Return the description as a single string
return desc.join(" ");
};
```

console

```
This histogram visualizes the frequency of hours children spend watching TV. 0 to 1 hours: 29 children. 1 to 2 hours: 47 children.
2 to 3 hours: 22 children. 3 to 4 hours: 13 children. 4 to 5 hours: 11 children. 5 to 6 hours: 9 children. 6 to 7 hours: 1
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

Keyboard settings

Screen readers use:

- Shift, Ctrl, Option, arrow and Tab keys for navigation
- Don’t override these with custom JavaScript

SWINBURNE UNIVERSITY OF TECHNOLOGY

Interactions

Users should be able to use keyboard to navigate page

In non-accessible code, keyboard users will not have access to tooltips



Non-keyboard accessible mouse over

```
const handleMouseEvents = () => {


    innerChart.selectAll("rect")
    .filter(function(d) { return d !== undefined; }) // because there are rectangles without data (ie t

            .on("mouseenter", (e, d) => {
              console.log(d);
              // Update the tooltip text with the bin's data
              d3.select(".tooltip text")
                .text(`${d.x0} to ${d.x1} hours: ${d.length} children`);


              // Get the rectangle's position and dimensions
              const rect = e.target;
              const rectX = parseFloat(rect.getAttribute("x")); // X position of the rectangle
              const rectY = parseFloat(rect.getAttribute("y")); // Y position of the rectangle
              const rectWidth = parseFloat(rect.getAttribute("width")); // Width of the rectangle
              const rectHeight = parseFloat(rect.getAttribute("height")); // Height of the rectangle

              // Calculate the center of the rectangle
              const tooltipX = rectX  // Center X of the rectangle
              const tooltipY = rectY  // Center Y of the rectangle

              // Position the tooltip in the middle of the rectangle
              d3.select(".tooltip")
                .attr("transform", `translate(${tooltipX}, ${tooltipY})`)
                .transition()
                  .duration(200)
                  .style("opacity", 1); // Fade in the tooltip
            })
            .on("mouseleave", () => {
              // Hide the tooltip when the mouse leaves the rectangle
              d3.select(".tooltip")
                .transition()
                  .duration(200)
                  .style("opacity", 0); // Fade out the tooltip
            });

}
```

SWINBURNE UNIVERSITY OF TECHNOLOGY

Interactions

Using keyboard to interact with chart and get access to tool tips:

Set up tool tips (as per previously)

In **handleMouseEvents()**:

- Create screen reader announcements
- Announce tooltip information to screen reader
- Allow mouse and keyboard events to trigger same behaviours

SWINBURNE UNIVERSITY OF TECHNOLOGY

handleMouseEvents

For line chart circles

```
const handleMouseEvents = () => {

    innerChart.selectAll("rect")
    .filter(function(d) { return d !== undefined; }) // because there are rectangles without data

        .on("mouseenter", (e, d) => {
          console.log(d);
          // Update the tooltip text with the bin's data
          d3.select(".tooltip text")
            .text(`${d.x0} to ${d.x1} hours: ${d.length} children`);


          // Get the rectangle's position and dimensions
          const rect = e.target;
          const rectX = parseFloat(rect.getAttribute("x")); // X position of the rectangle
          const rectY = parseFloat(rect.getAttribute("y")); // Y position of the rectangle

          // Calculate the center of the rectangle
          const tooltipX = rectX  // Center X of the rectangle
          const tooltipY = rectY  // Center Y of the rectangle

          // Position the tooltip in the middle of the rectangle
          d3.select(".tooltip")
            .attr("transform", `translate(${tooltipX}, ${tooltipY})`)
            .transition()
              .duration(200)
              .style("opacity", 1); // Fade in the tooltip
        })
        .on("mouseleave", () => {
          // Hide the tooltip when the mouse leaves the rectangle
          d3.select(".tooltip")
            .transition()
              .duration(200)
              .style("opacity", 0); // Fade out the tooltip
        });

}
```

GitHub Copilot

```
const createTooltip = () => {

  const tooltip = innerChart
      .append("g")
      .attr("class", "tooltip")
      .style("opacity", 0);

  tooltip
  .append("rect")
      .attr("width", tooltipWidth)
      .attr("height", tooltipHeight)
      .attr("rx", 3)
      .attr("ry", 3)
      .attr("fill", aubergine)
      .attr("fill-opacity", 1);

  tooltip
  .append("text")
      .attr("x", tooltipWidth/2)
      .attr("y", tooltipHeight/2 + 5)
      .attr("text-anchor", "middle")
      .attr("alignment-baseline", "middle")
      .attr("fill", "white")
      .style("font-weight", 900);
}
```

Keyboard accessible histogram

```
const handleMouseEvents = () => {

  // Create a live region for screen reader announcements if it doesn't exist
  if (!document.getElementById("tooltip-announcer")) {
    const announcer = document.createElement("div");
    announcer.id = "tooltip-announcer";
    announcer.setAttribute("aria-live", "polite");
    announcer.className = "sr-only"; // Style this with CSS to be visually hidden
    document.body.appendChild(announcer);
  }

  // Shared function to show tooltip (used by both mouse and keyboard events)
  const showTooltip = (e, d) => {
    // Update the tooltip text
    d3.select(".tooltip text")
      .text(`${d.x0} to ${d.x1} hours: ${d.length} children`);

    // Position the tooltip
    const rect = e.target;
    const rectX = parseFloat(rect.getAttribute("x"));
    const rectY = parseFloat(rect.getAttribute("y"));

    d3.select(".tooltip")
      .attr("transform", `translate(${rectX}, ${rectY})`)
      .transition()
        .duration(200)
        .style("opacity", 1);

    // Announce to screen readers
    document.getElementById("tooltip-announcer").textContent =
      `${d.x0} to ${d.x1} hours: ${d.length} children`;
  };

  // Shared function to hide tooltip
  const hideTooltip = () => {
    d3.select(".tooltip")
      .transition()
        .duration(200)
        .style("opacity", 0);
  };

  // Select all data-bound rectangles
  innerChart.selectAll("rect")
    .filter(function(d) { return d !== undefined; })
    .attr("tabindex", 0)
    .attr("aria-label", d => `Bar representing ${d.x0} to ${d.x1} hours with ${d.length} children`)

    // Mouse events
    .on("mouseenter", showTooltip)
    .on("mouseleave", hideTooltip)

    // Keyboard events (for accessibility)
    .on("focus", showTooltip)  // Show tooltip when focused with Tab key
    .on("blur", hideTooltip)   // Hide tooltip when focus moves away

    // Add keyboard navigation between bars
    .on("keydown", (e, d) => {
      // Navigate between bars with arrow keys
      if (e.key === "ArrowRight" || e.key === "ArrowDown") {
        e.preventDefault();
        const nextBar = e.target.nextElementSibling;
        if (nextBar && nextBar.tagName.toLowerCase() === "rect") {
          nextBar.focus();
        }
      } else if (e.key === "ArrowLeft" || e.key === "ArrowUp") {
        e.preventDefault();
        const prevBar = e.target.previousElementSibling;
        if (prevBar && prevBar.tagName.toLowerCase() === "rect") {
          prevBar.focus();
        }
      }
    });
}
```

SWINBURNE UNIVERSITY OF TECHNOLOGY

Create screen reader announcement

Functions to show and hide tool tip and announce to screen reader

Allow mouse and keyboard to trigger events

```
const handleMouseEvents = () => {

  // Create a live region for screen reader announcements if it doesn't exist
  if (!document.getElementById("tooltip-announcer")) {
    const announcer = document.createElement("div");
    announcer.id = "tooltip-announcer";
    announcer.setAttribute("aria-live", "polite");
    announcer.className = "sr-only"; // Style this with CSS to be visually hidden
    document.body.appendChild(announcer);
  }

  // Shared function to show tooltip (used by both mouse and keyboard events)
  const showTooltip = (e, d) => {
    // Update the tooltip text
    d3.select(".tooltip text")
      .text(`${d.x0} to ${d.x1} hours: ${d.length} children`);

    // Position the tooltip
    const rect = e.target;
    const rectX = parseFloat(rect.getAttribute("x"));
    const rectY = parseFloat(rect.getAttribute("y"));

    d3.select(".tooltip")
      .attr("transform", `translate(${rectX}, ${rectY})`)
      .transition()
        .duration(200)
        .style("opacity", 1);

    // Announce to screen readers
    document.getElementById("tooltip-announcer").textContent =
      `${d.x0} to ${d.x1} hours: ${d.length} children`;
  };

  // Shared function to hide tooltip
  const hideTooltip = () => {
    d3.select(".tooltip")
      .transition()
        .duration(200)
        .style("opacity", 0);
  };

  // Select all data-bound rectangles
  innerChart.selectAll("rect")
    .filter(function(d) { return d !== undefined; })
    .attr("tabindex", 0)
    .attr("aria-label", d => `Bar representing ${d.x0} to ${d.x1} hours with ${d.length} children`)

    // Mouse events
    .on("mouseenter", showTooltip)
    .on("mouseleave", hideTooltip)

    // Keyboard events (for accessibility)
    .on("focus", showTooltip)  // Show tooltip when focused with Tab key
    .on("blur", hideTooltip)   // Hide tooltip when focus moves away

    // Add keyboard navigation between bars
    .on("keydown", (e, d) => {
      // Navigate between bars with arrow keys
      if (e.key === "ArrowRight" || e.key === "ArrowDown") {
        e.preventDefault();
        const nextBar = e.target.nextElementSibling;
        if (nextBar && nextBar.tagName.toLowerCase() === "rect") {
          nextBar.focus();
        }
      } else if (e.key === "ArrowLeft" || e.key === "ArrowUp") {
        e.preventDefault();
        const prevBar = e.target.previousElementSibling;
        if (prevBar && prevBar.tagName.toLowerCase() === "rect") {
          prevBar.focus();
        }
      }
    });
}
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

Screen reader announcement

```
// Create a live region for screen reader announcements if it doesn't exist
if (!document.getElementById("tooltip-announcer")) {
    const announcer = document.createElement("div");
        announcer.id = "tooltip-announcer";
        announcer.setAttribute("aria-live", "polite");
        announcer.className = "sr-only"; // Style this with CSS to be visually hidden
        document.body.appendChild(announcer);
}
```

CSS

```
.sr-only {
position: absolute;
width: 1px;
height: 1px;
padding: 0;
margin: -1px;
overflow: hidden;
clip: rect(0, 0, 0, 0);
white-space: nowrap;
border: 0;
}
```

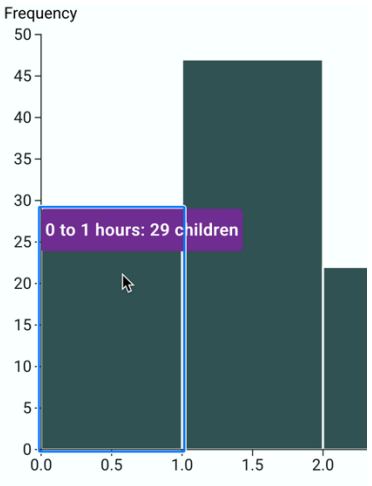
GitHub Copilot

SWINBURNE UNIVERSITY OF TECHNOLOGY

Show tool tip

```
// Shared function to show tooltip (used by both mouse and keyboard events)
const showTooltip = (e, d) => {
// Update the tooltip text
    d3.select(".tooltip text")
    .text(`${d.x0} to ${d.x1} hours: ${d.length} children`);
// Position the tooltip
    const rect = e.target;
    const rectX = parseFloat(rect.getAttribute("x"));
    const rectY = parseFloat(rect.getAttribute("y"));
    d3.select(".tooltip")
        .attr("transform", `translate(${rectX}, ${rectY})`)
        .transition()
        .duration(200)
        .style("opacity", 1);
// Announce to screen readers
    document.getElementById("tooltip-announcer").textContent =
    `${d.x0} to ${d.x1} hours: ${d.length} children`;
};
```

No offset in this tool tip code



GitHub Copilot

Hide Tool tip

```
// Shared function to hide tooltip
const hideTooltip = () => {
    d3.select(".tooltip")
    .transition()
    .duration(200)
    .style("opacity", 0);
};
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

Allow mouse and keyboard to trigger event

```
// Select all data-bound rectangles
innerChart.selectAll("rect")
      .filter(function(d) { return d !== undefined; })
      .attr("tabindex", 0)
      .attr("aria-label", d => `Bar representing ${d.x0} to ${d.x1} hours with ${d.length} children`)

// Mouse events
      .on("mouseenter", showTooltip)
      .on("mouseleave", hideTooltip)

// Keyboard events (for accessibility)
      .on("focus", showTooltip) // Show tooltip when focused with Tab key
      .on("blur", hideTooltip) // Hide tooltip when focus moves away
```

Don't try select tool tip rectangles (i.e., rectangles with no data)!

SWINBURNE UNIVERSITY OF TECHNOLOGY

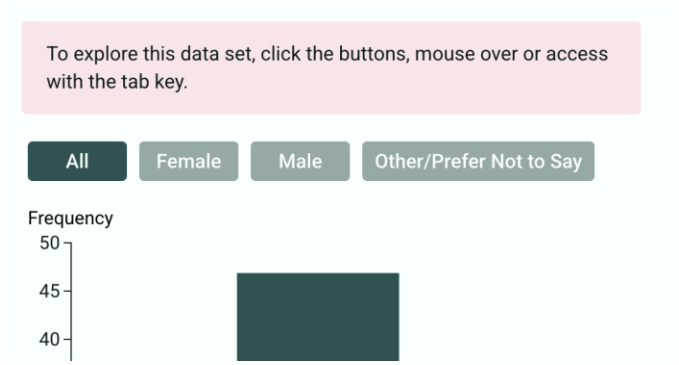
Allow mouse and keyboard to trigger event

```
// Add keyboard navigation between bars
.on("keydown", (e, d) => {
// Navigate between bars with arrow keys
     if (e.key === "ArrowRight" || e.key === "ArrowDown") {
          e.preventDefault();
          const nextBar = e.target.nextElementSibling;
          if (nextBar && nextBar.tagName.toLowerCase() === "rect") {
          nextBar.focus();
          }
     } else if (e.key === "ArrowLeft" || e.key === "ArrowUp") {
          e.preventDefault();
          const prevBar = e.target.previousElementSibling;
          if (prevBar && prevBar.tagName.toLowerCase() === "rect") {
          prevBar.focus();
          }
     }
});
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

Interaction cues (index.html)...

```
<div class="instructions">To explore this data set, click the buttons, mouse over or access with the tab key.</div>
```



CSS

```
.instructions {
    max-width: 500px;
    margin-top: 30px;
    padding: 15px 20px 15px 20px;
    background-color: rgba(239, 169, 174, 0.3);
    background-repeat: no-repeat;
    background-position: 15px center;
    background-size: auto 25px;
    border-radius: 4px;
}
@media (max-width: 700px) {
    .instructions {
        margin-bottom: 10px;
}
```

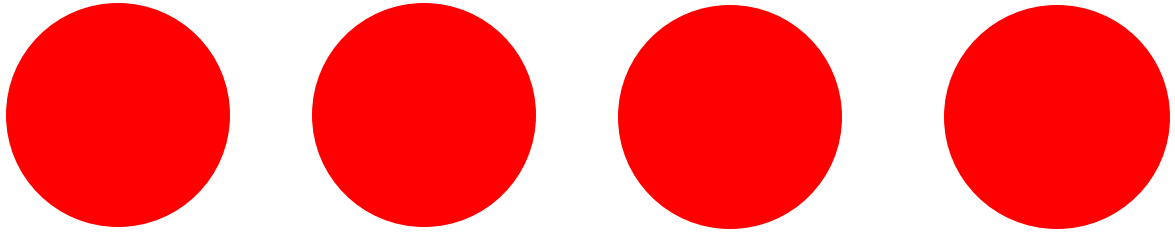

SWINBURNE UNIVERSITY OF TECHNOLOGY

Other considerations

Some users import own style sheet

- Be careful overriding user settings by use of inline **style()** in D3

Avoid flashing red (trigger epileptic fit)



SWINBURNE UNIVERSITY OF TECHNOLOGY